

使用 Cloud Shell 和 Cloud Code 進行開發

來源：<https://codelabs.developers.google.com/cloudshell-cloudcode-deepdive?hl=zh-tw>

步驟數：6

瞭解 Cloud Shell 和 Cloud Shell 編輯器的功能與特色。

目錄

1. [總覽](#)
2. [設定和需求](#)
3. [使用 Cloud Shell 終端機](#)
4. [使用 Cloud Shell 編輯器](#)
5. [使用 Cloud Code](#)
6. [恭喜！](#)

1. 總覽

Cloud Shell 是一套線上開發與作業環境，可透過瀏覽器隨時隨地存取。Cloud Shell 的線上終端機已預先載入 `kubectl`、`gcloud` 指令列工具等公用程式，方便您管理資源。您也可以使用線上 Cloud Shell 編輯器開發、建構及部署雲端式應用程式，並為應用程式進行偵錯

在本實驗室中，您將使用 Cloud Shell 和 Cloud Shell 編輯器，透過 Cloud Code 在本機模擬器和實際服務上，建立、部署及測試以容器為基礎的應用程式。

學習目標

- 瀏覽並熟悉 Cloud Shell 的主要功能
- 練習使用不同的 Cloud Shell 用法模式
- 自訂 Cloud Shell 環境以供進階用途
- 熟悉 Cloud Code 選項和功能
- 瞭解 Kubernetes 應用程式的 Cloud Code 詳細資料
- 使用 Minikube 等本機模擬器

必要條件

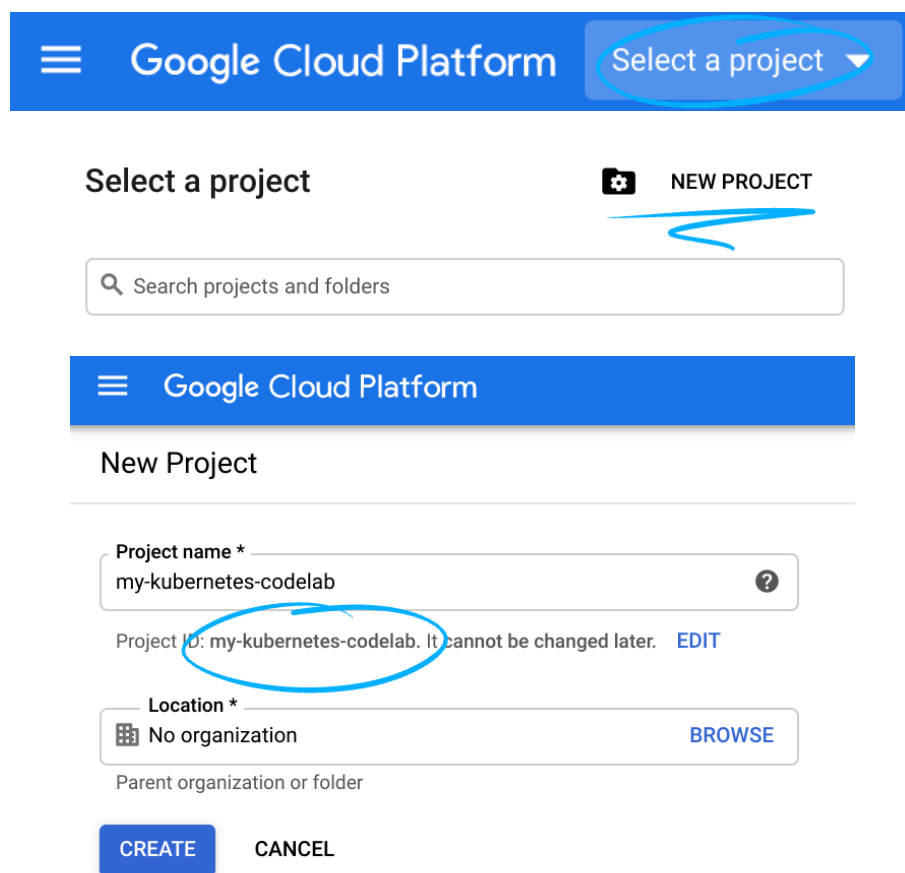
- 您需要具備編輯者權限的 GCP 專案、GCP 帳戶和 Cloud Shell 存取權
 - 如要完成選用部分，您需要安裝終端機模擬器和 Google Cloud SDK。
-

步驟 2 · 約 0 分鐘

2. 設定和需求

自修實驗室環境設定

1. 登入 [Google Cloud 控制台](#)，然後建立新專案或重複使用現有專案。如果沒有 Gmail 或 Google Workspace 帳戶，請先[建立帳戶](#)。



- **專案名稱**是這個專案參與者的顯示名稱。這是 Google API 未使用的字元字串。你隨時可以更新該位置資訊。
- **專案 ID** 在所有 Google Cloud 專案中都是不重複的，而且設定後即無法變更。Cloud 控制台會自動產生不重複的字串，通常您不需要在意這個字串。在大多數程式碼研究室中，您需要參照專案 ID (通常會標示為 `PROJECT_ID`)。如果您不喜歡產生的 ID，可以產生另一個隨機 ID。你也可以嘗試自訂名稱，看看是否可用。完成這個步驟後就無法變更，且專案期間都會維持這個設定。
- 請注意，部分 API 會使用第三個值，也就是「專案編號」。如要進一步瞭解這三種值，請參閱[說明文件](#)。

注意：專案 ID 在全域中是專屬 ID，選定後就無法變更，且其他使用者無法使用。您是該 ID 的唯一使用者。即使刪除專案，該 ID 也無法再使用

注意：如果您使用 Gmail 帳戶，可以將預設位置設為「沒有機構」。如果您使用 Google Workspace 帳戶，請選擇適合貴機構的位置。

2. 接著，您需要在 Cloud 控制台中[啟用帳單](#)，才能使用 Cloud 資源/API。完成本程式碼研究室的費用應該不高，甚至完全免費。如要關閉資源，避免產生本教學課程以外的費用，您可以刪除自己建立的資源，或刪除整個專案。Google

Cloud 新使用者可參加[價值\\$300 美元的免費試用計畫](#)。

步驟 3 · 約 0 分鐘

3. 使用 Cloud Shell 終端機

本節將介紹 Cloud Shell 終端機、如何瀏覽 UI、使用各項功能，以及探索設定選項和各種使用方式，協助您提高工作效率。

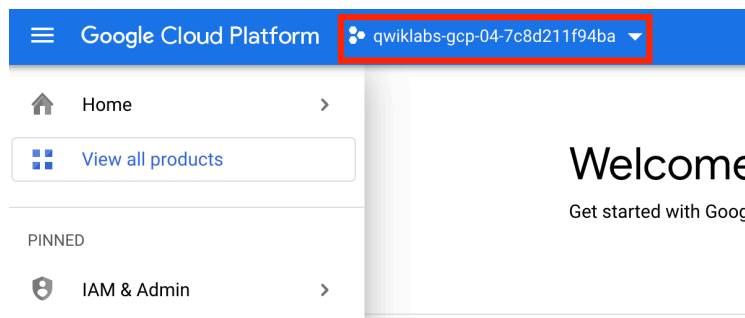
Cloud Shell 有兩項主要工具：**Cloud Shell 終端機**和 **Cloud Shell 編輯器**。在本實驗室中，「Cloud Shell 終端機」和「Cloud Shell」這兩個詞彙會交替使用，但「編輯器」一律稱為「Cloud Shell 編輯器」，以清楚區分終端機模擬器和 IDE。

Cloud Shell 是功能齊全的雲端 Shell，以暫時性的 Google Compute Engine Debian Linux 機器為基礎，隨時可供您使用。並預先安裝 Google Cloud SDK 和多種相關工具，例如 `git`、`kubectl`、`kubectx`、`curl`、`python3`、`tmux` ...

Google Cloud 會將 `$HOME` 目錄保留在 Cloud Storage 中，並在每次啟動 Cloud Shell 電腦時自動掛接，因此即使 Cloud Shell 電腦關閉，您也不會遺失任何儲存在該目錄下的資訊。也就是說，如果您再次啟用 Cloud Shell，系統會保留 `$HOME` 目錄和您對該目錄所做的任何自訂設定，供您繼續使用。

啟動 Cloud Shell

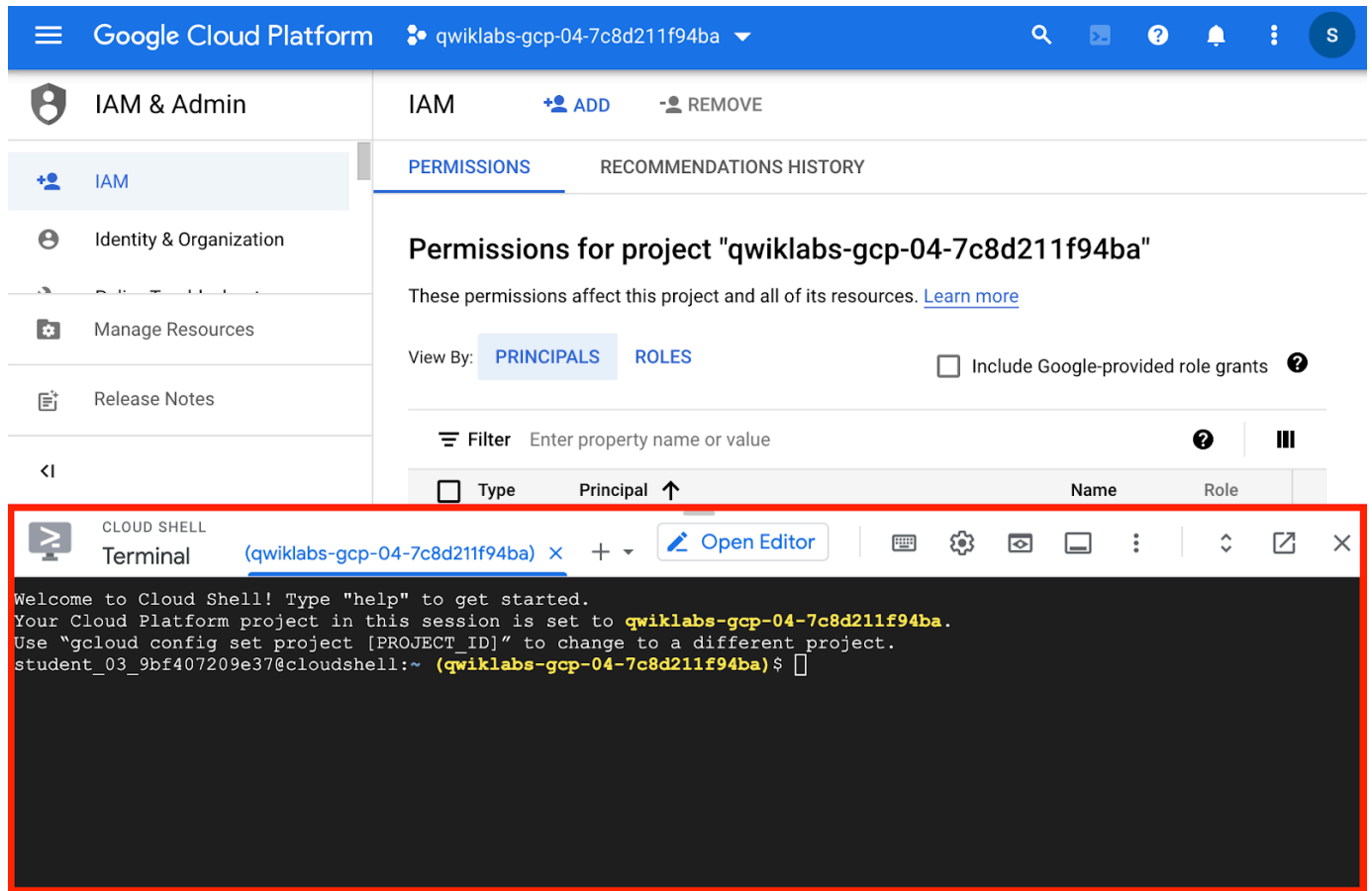
1. 前往 `console.cloud.google.com`，如果尚未登入 GCP 主控台，請輸入 Google 憑證。系統會顯示 Google Cloud Platform 的主控台主要 UI。
2. 在 Cloud 控制台選單列選取器中，選取要使用的 GCP 專案 ID：



3. 在選單列右側，帳戶的虛擬人偶旁邊，找到啟用 Cloud Shell 的按鈕：



4. 按一下 Cloud Shell 圖示，開啟 Cloud Shell 終端機，並在 GCP Console UI 下方顯示新窗格：



這個窗格就是所謂的 Cloud Shell 終端機。

5. 前往 Cloud Shell 終端機，並注意您從 GCP 主控台選取適當的 GCP 專案 ID 時，Cloud Shell 會知道該 ID，因此您不必再次指定：

```
echo $GOOGLE_CLOUD_PROJECT
```

這會輸出您的 GCP 專案 ID，也會顯示在 Cloud Shell 提示中。

切換工具

Cloud Shell 提供多種工具和設定選項。透過 Cloud Shell 選單中的選項查看：



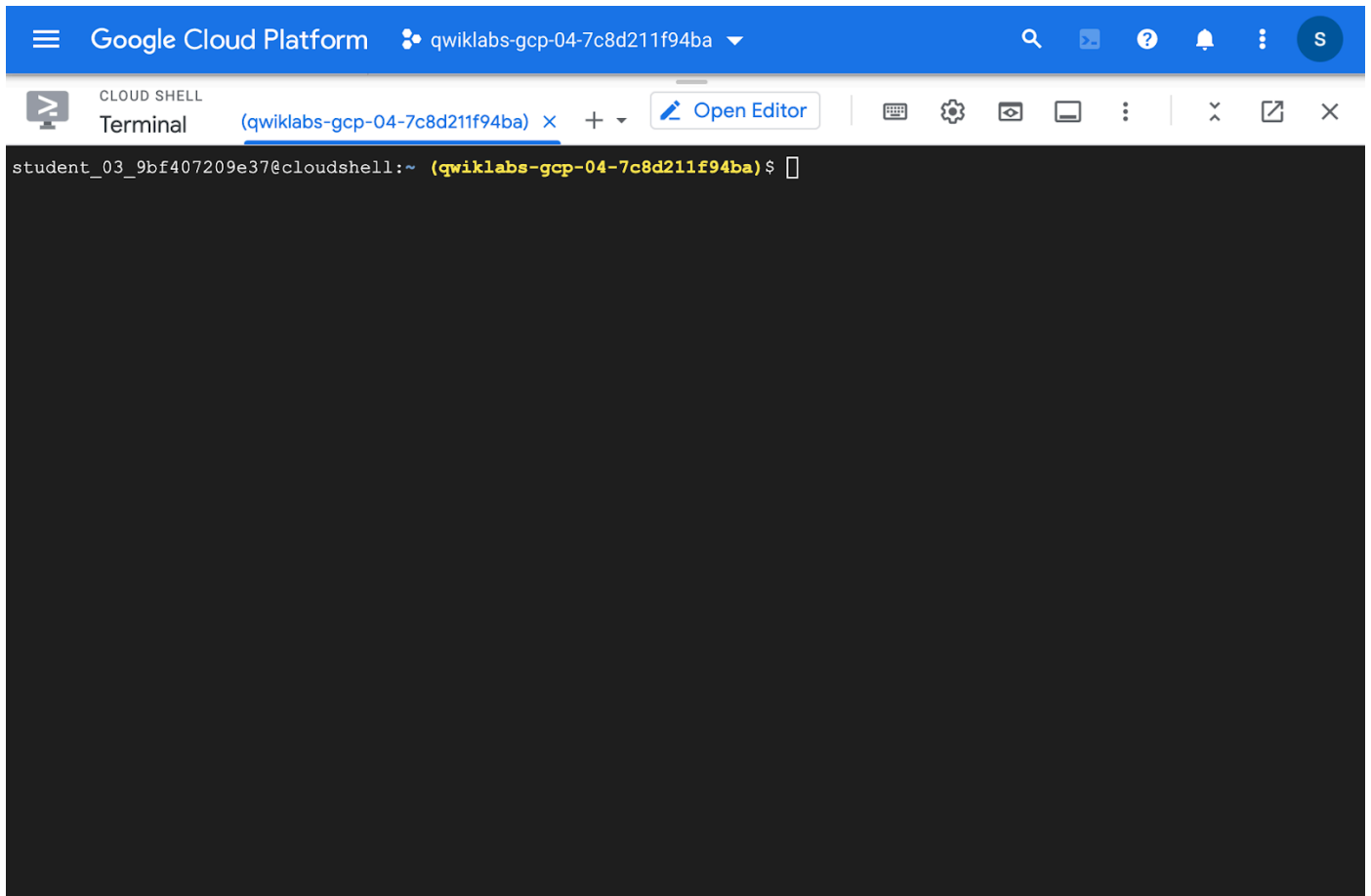
1. 按一下工具列中的



按鈕，即可最大化 Cloud Shell 終端機，並佔用瀏覽器分頁的所有空間：



2. Cloud Shell 會盡量擴大並佔用整個瀏覽器分頁空間：



3. 再次點選該圖示，瀏覽器分頁會恢復為 Cloud 控制台和 Cloud Shell 共用。

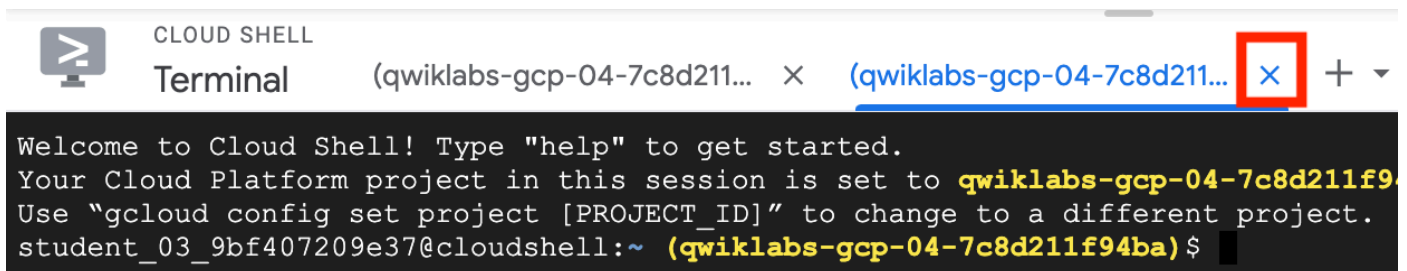
開啟新分頁

您可以在 Cloud Shell 終端機中開啟任意數量的分頁。

1. 按一下下列按鈕，開啟新分頁。



2. 現在請關閉新分頁，方法是點選分頁名稱旁的 x，或在對應的殼層中輸入 `exit`：



開啟新視窗

在新瀏覽器視窗中開啟 Cloud Shell，即可在一個視窗中開啟 Cloud Shell，另一個視窗則開啟 Cloud 控制台。

1. 點選下列按鈕，觀察 Cloud Shell 如何在新瀏覽器分頁中開啟：



2. 請注意，Cloud Shell 現在提供相同的工具列選項，但外觀略有不同：



3. 請在後續章節中保持開啟這個新的瀏覽器分頁。

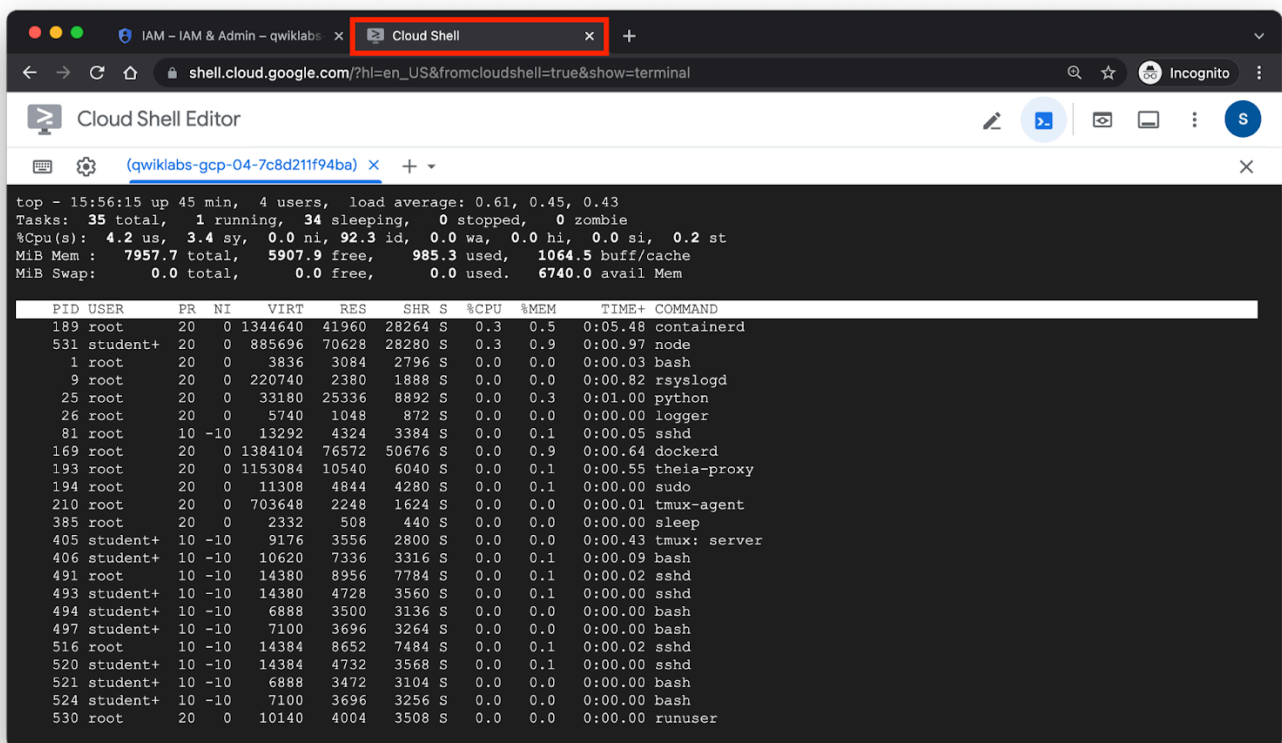
使用 Tmux 轉移工作階段

Cloud Shell 內含 [tmux](#)。Tmux 是非常熱門的終端機多工器，類似於 GNU Screen。Cloud Shell 與 tmux 整合後，無論您身在何處，都能保留工作階段。

在下列步驟中，您將練習使用這項概念，進一步瞭解這項功能。

移動 Cloud Shell 終端機

1. 在剛開啟的 Cloud Shell 分頁中，輸入 `top` 指令：

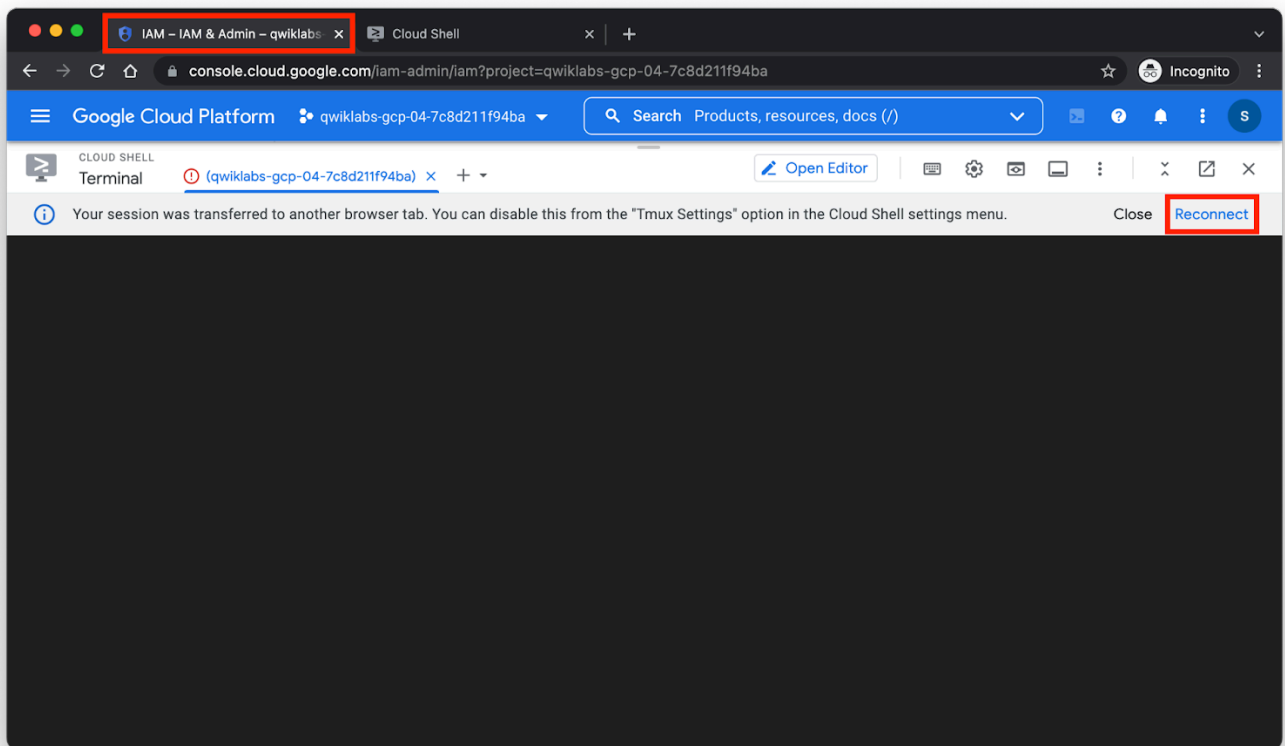


`top` 指令會即時顯示在 Cloud Shell 終端機中執行的程序。您將使用這項功能，以視覺化方式呈現 tmux 提供的 Cloud Shell 工作階段持續性。

2. 返回瀏覽器的第一個分頁 (Cloud 控制台所在的分頁)。

3. 請注意，由於您先前決定在新視窗中開啟 Cloud Shell，因此會看到「Your session was transferred to another browser tab. You can disable this from the "Tmux Settings" option in the Cloud Shell settings menu.」訊息

4. 按一下「重新連線」按鈕：



接著，您就能存取在第二個分頁中執行的殼層，而 `top` 會繼續執行，就像您剛才離開時一樣。

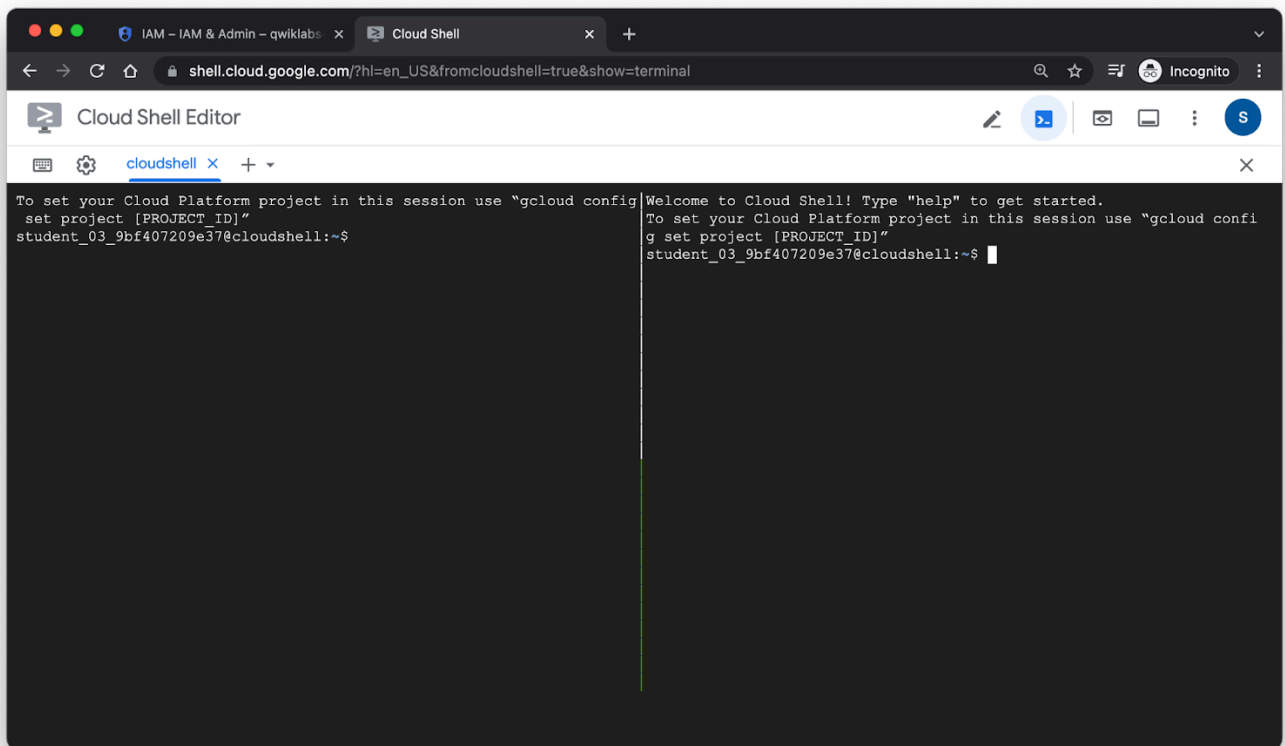
5. 前往瀏覽器中的第二個分頁，您已在新視窗中開啟 Cloud Shell，
6. 按一下「重新連結」，您會復原頂端程序，就像我們關閉視窗前一樣。

使用 Tmux 分割視窗

Tmux 的用途遠遠不只如此。本實驗室不會介紹 tmux 教學課程，如要進一步瞭解，請參閱 [tmux 官方入門指南](#)。

不過，tmux 為 Cloud Shell 提供一些額外功能，這些功能在本地終端機模擬器中也可能存在。如要示範，請並排分割終端機。

1. 在 Cloud Shell 中，依序按下 `Ctrl + b` 和 `%`
2. 觀察 tmux 如何在原始窗格右側建立新窗格：



3. 按一下第二個窗格，然後輸入 `exit`。

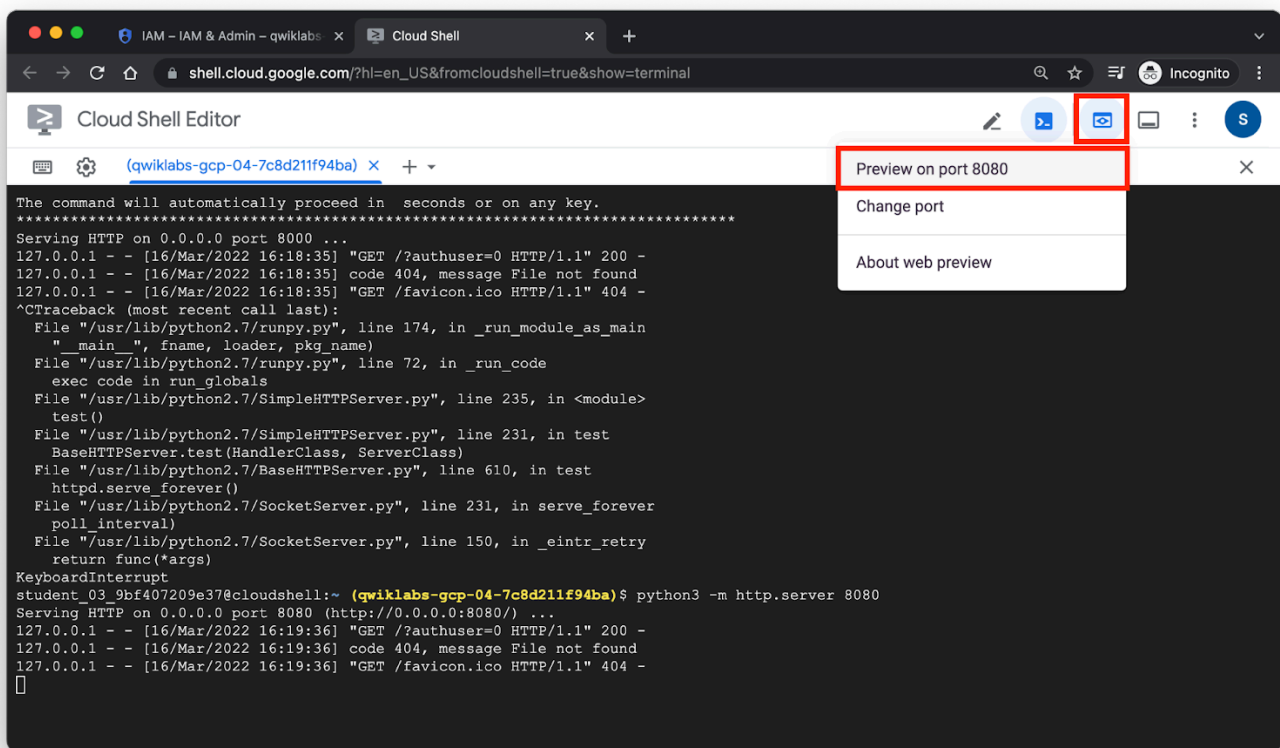
使用網頁預覽

Cloud Shell 也能透過電腦上執行的本機瀏覽器，從雲端執行的 Cloud Shell 電腦建立自動連接埠轉送。

1. 在 Cloud Shell 中輸入：

```
python3 -m http.server 8080
```

2. 前往右上角的「網頁預覽」選項，然後選取「透過以下通訊埠預覽：8080」：



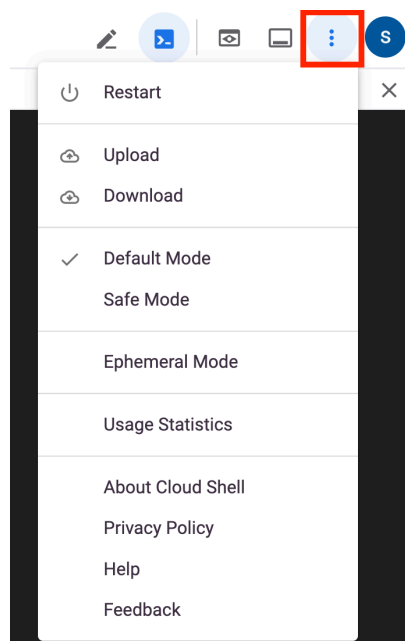
系統會開啟新分頁，顯示該連接埠提供的預設頁面。

請注意，您可以在 Cloud Shell 中變更通訊埠，並預覽應用程式公開的任何通訊埠，不限於通訊埠 `8080`。

存取其他 Cloud Shell 選項

Cloud Shell 提供其他設定和選項。

1. 按一下 Cloud Shell 選單中的三點圖示，即可存取下列選項：



這裡最相關的選項是上傳或下載檔案至 Cloud Shell，以及在發生任何問題時重新啟動 Cloud Shell 電腦。

4. 使用 Cloud Shell 編輯器

Cloud Shell 最重要的功能之一是 Cloud Shell 編輯器。這是以開放原始碼專案 [Eclipse Theia](#) 為基礎的完整 IDE，Google 等廠商也參與了開發。兩者都使用多個常見的開放原始碼基本元件，例如 Monaco 編輯器、擴充功能模型、語言伺服器通訊協定和偵錯介面卡通訊協定，因此外觀和風格非常相似。

由於 Theia/Cloud Shell 編輯器是相當複雜的工具，因此本實驗不會說明所有 Cloud Shell 編輯器功能。在本節中，您將瞭解 Google Cloud 實作方案的一些核心概念和獨特功能。

存取 Cloud Shell 編輯器

您可以透過三種方式存取 Cloud Shell 編輯器：

- 從指令列存取
- 透過 Cloud Shell 選單存取
- 在瀏覽器中前往網址

請按照下列步驟體驗這些方法

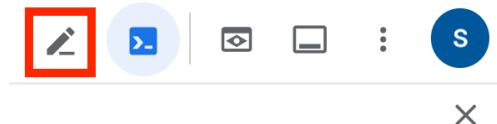
1. 使用下列指令，從指令列開啟 `.bashrc`：

```
cloudshell edit $HOME/.bashrc
```

2. 從選單開啟 **Cloud Shell**，然後按一下「Open Editor」（開啟編輯器）



按鈕：



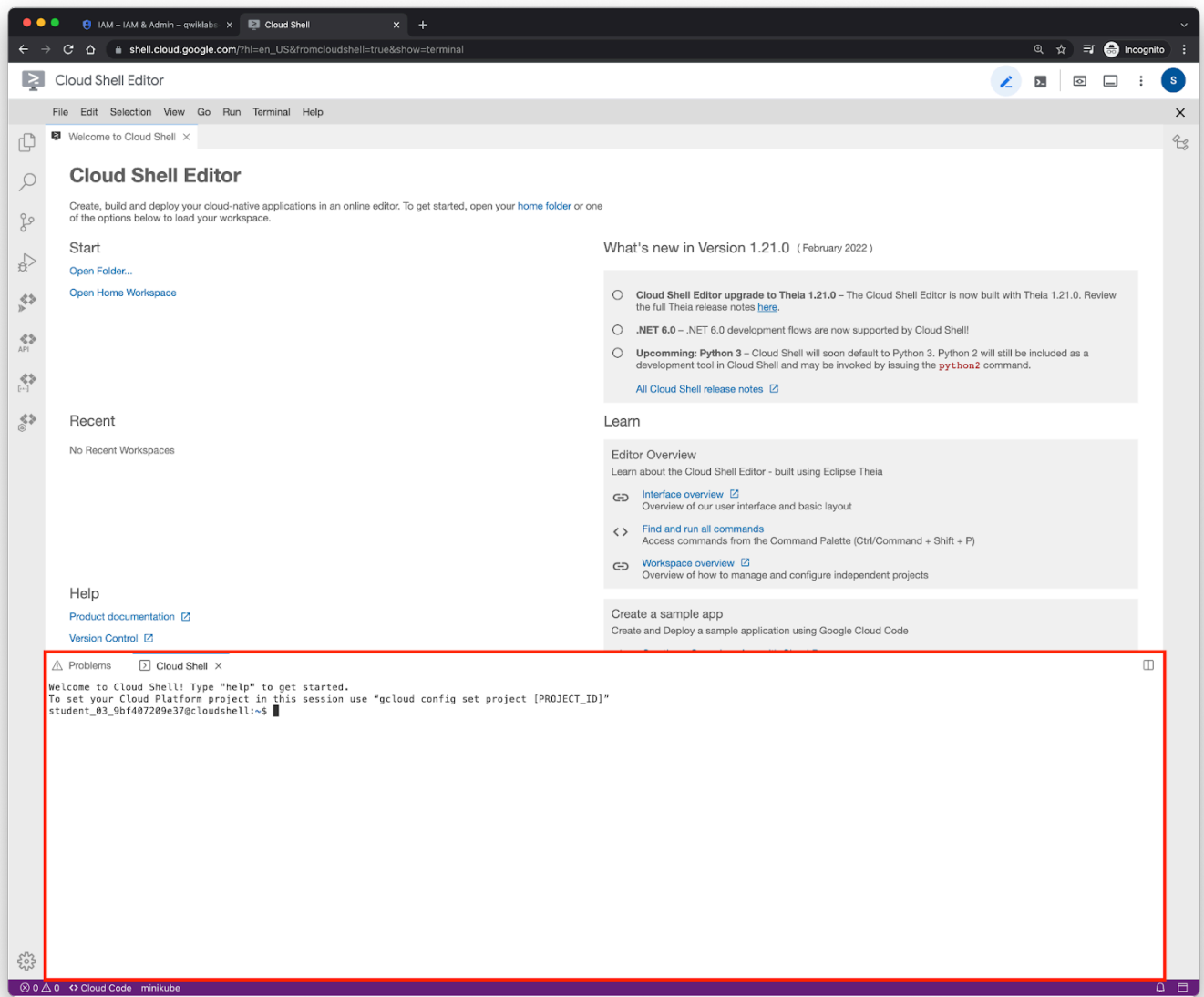
這樣會開啟 Cloud Shell 編輯器，同時保持 Cloud Shell 終端機開啟。如要將所有分頁空間都用於 Cloud Shell 編輯器，請按一下「編輯」按鈕右側的按鈕，關閉終端機。這兩個按鈕（開啟/關閉編輯器和開啟/關閉終端機）會決定瀏覽器分頁中顯示的工具。請練習點選這兩者，掌握訣竅。

3. 在瀏覽器的網址列中輸入 `ide.cloud.google.com`，即可叫用 Cloud Shell 編輯器。

使用 Cloud Shell 編輯器的指令列

雖然 Cloud Shell 終端機和 Cloud Shell 編輯器會共存在同一個瀏覽器分頁中，但您可以捨棄 Cloud Shell 終端機，只使用 Cloud Shell 編輯器，以及 Cloud Shell 編輯器內建的終端機，滿足所有需求。如果您習慣在 IDE 中工作，且熟悉 VSCode 的各種快速鍵和窗格動態，這或許是個不錯的選擇。

使用上述任一方法開啟 Cloud Shell 編輯器。然後前往「編輯器」選單，依序選取「Terminal」>「New Terminal」（或按鍵盤上的 `Ctrl + ``）。Cloud Shell 編輯器下方會開啟新的終端機窗格，您可以在這裡執行所有 Shell 作業：

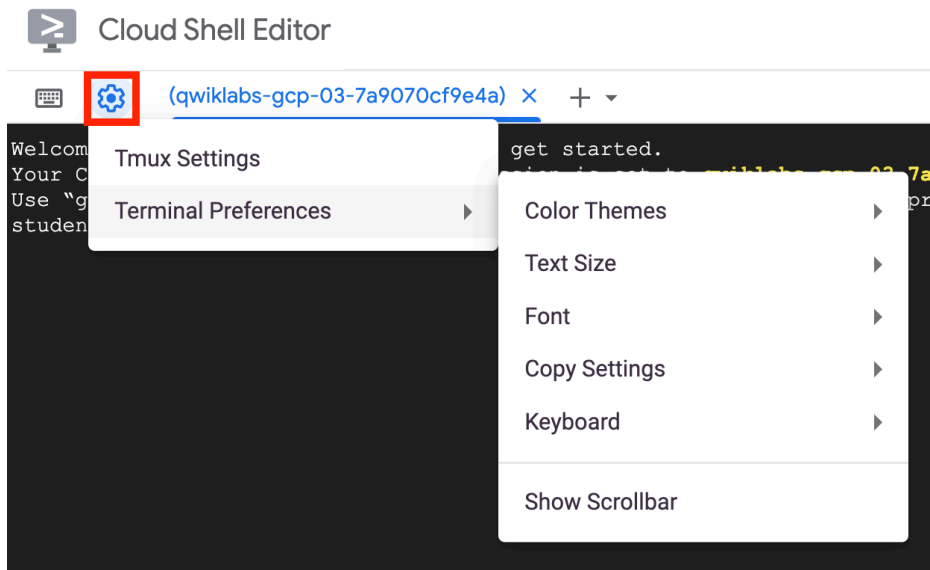


您也可以藉此管理不同的終端機窗格，不必像 Cloud Shell 終端機一樣使用 tmux 窗格管理。

自訂 Cloud Shell 設定

Cloud Shell 提供多種自訂選項，包括外觀和基本行為，以及 Cloud Shell 終端機啟動設定和詳細操作。您將在以下各節中詳細瞭解不同選項。

前往 Cloud Shell 終端機選單，然後按一下齒輪圖示。畫面上會顯示多個設定選項：



請參閱以下幾項：

- **色彩主題：**你可以使用淺色、深色，或一組自訂顏色 (基本顏色，例如字型顏色和背景顏色)
- **文字大小：**5 種不同字型大小可供選擇
- **字型：**Courier New 或 Monospace
- **複製設定：**您可以在這裡變更複製鍵盤快速鍵，使其與最熱門的 Linux 終端機模擬器所用的快速鍵類似。
- **鍵盤：**將 Meta 鍵對應至 Alt (預設為 ESC)，以及在某些作業系統中對應 Alt Gr 鍵。

設定 Shell 環境

在以 Debian 為基礎的 Linux 電腦中，終端機模擬器會執行殼層，因此您可以像在 Linux 中一樣自訂殼層環境。因此，每次建立新的登入殼層程序時，系統都會從 `$HOME` 取得任何設定檔 (例如 `.bashrc`)。

此外，您也可以名為 `.customize_environment` 的特殊設定檔中，指定每次啟動 Cloud Shell 執行個體時要執行的動作。詳情請參閱「[環境自訂](#)」說明文件。

練習在 Cloud Shell 終端機中新增一些設定。前往 Cloud Shell 終端機，然後複製下列範例程式碼存放區：

```
git clone https://gitlab.com/javiercanadillas/cloud-code-getting-started.git
```

這個存放區包含兩個主要事件的範例，您可能需要為 Cloud Shell 設定這些事件：

- 每當 Cloud Shell 執行個體啟動時，系統都會讀取 `$HOME/.customize_environment` 檔案。您可以在 `/var/log/customize_environment` 中找到這項程序的執行記錄，成功執行後會建立 `/google/devshell/customize_environment_done`。
- 每當啟動新的殼層程序 (新視窗、新分頁等)，系統就會讀取一般的 Bash 殼層設定檔。

請按照下列步驟，練習使用這兩種方法：

1. 執行下列指令，啟動存放區提供的自訂項目：

```
cd cloud-code-getting-started
source set_env_cust.sh
```

這會設定並啟用上述兩種自訂選項。

2. 請注意新的殼層提示。 `.bash_profile` 檔案中提供的 Bash 指令碼已設定這個新提示，比預設提示短，且包含基本 Git 資訊，例如已簽出的分支或 Git 狀態。
3. 現在 `cloudshell edit` 指令有新的 **code** 別名，長度較短。使用這個指令查看 `.bash_profile` 檔案的內容：

```
code $HOME/.bash_profile
```

4. 使用新安裝的 `bat` 指令查看 `.customize_environment` 檔案內容：

```
bat $HOME/.customize_environment
```

`bat` 是熱門 Unix 工具 `cat` 的強化版。這裡的 `bat` 輸出內容顯示，每次啟動新的 Cloud Shell 機器時，都會使用 `apt install` 指令重新安裝負載測試工具 `hey`。

5. 按一下 Cloud Shell 選單 (右上角的三點圖示)，然後選取「重新啟動」，測試一切是否正常運作。

這樣一來，您就能模擬 Cloud Shell 超時和執行個體重建作業，確保所有設定正確無誤。

自訂 Cloud Shell 容器映像檔

這個自訂選項可讓您建立 Docker 映像檔，做為自訂 Cloud Shell 環境，並提供額外的套件和自訂設定。產生的映像檔會啟動完全暫時性的 Cloud Shell 執行個體，因此 Cloud Shell VM 和附加的 `$HOME` 目錄都不會保留。不過，如果您需要將特定功能的 Cloud Shell 執行個體提供給第三方，讓他們以最有效率的方式執行特定工作，那麼建立這類映像檔就很有用。

您不會像上一節一樣自訂環境，而是將變更內容烘焙到新映像檔，用於啟動 Cloud Shell。這項功能顯然有許多優點，因為自訂 Cloud Shell 的啟動速度會更快。

建立新的 Cloud Shell 映像檔

1. 在 Cloud Shell 終端機中輸入下列內容，開始建立 Cloud Shell 容器映像檔：

```
cloudshell env create-custom-image custom-cloud-shell  
cd custom-cloud-shell
```

這會在 Cloud Shell 中建立新目錄，並建立同名的新 Cloud Source Repositories 存放區來存程式碼。同時也會複製 `$HOME/custom-cloud-shell/ directory` 中的範例 `Dockerfile`。

2. 在 `Dockerfile` 的最後一行新增 `RUN apt install -y hey`：

```
echo "RUN apt install -y hey" >> $HOME/custom-cloud-shell/Dockerfile
```

這會設定與 `.customize_environment` 相同的自訂項目，但會改為內建在容器中。

3. 在本機建構映像檔：

```
cloudshell env build-local
```

4. 執行下列指令來測試新映像檔：

```
cloudshell env run
```


您現在位於映像檔內的殼層。

5. 執行 `hey` 指令，測試是否已安裝：

```
hey
```

6. 完成後，輸入 `exit` 退出容器：

```
exit
```

7. 將變更推送到 Cloud Source Repository，並將映像檔推送到 Container Registry：

```
git commit -a -m "Initial commit"
git push origin master
cloudshell env push
```

測試新圖片

1. 公開發布映像檔，方便您使用：

```
gsutil iam ch allUsers:objectViewer $(gsutil ls)
```

2. 產生可用於測試已發布 Cloud Shell 自訂執行個體的網址：

```
echo "https://ssh.cloud.google.com/cloudshell/editor?
cloudshell_image=gcr.io/$GOOGLE_CLOUD_PROJECT/custom-cloud-shell"
```

3. 複製輸出網址並貼到新的瀏覽器分頁，即可開啟 Cloud Shell 自訂執行個體。請注意，存取執行個體時，系統會顯示橫幅，指出您是以完整暫時模式執行：



Cloud Shell Editor

EPHEMERAL

4. 再次執行 `hey` 指令來測試圖片

```
hey
```

5. 完成後，請 `exit` 關閉暫時性 Cloud Shell 執行個體，返回開啟一般 Cloud Shell 的分頁，然後按一下「重新連線」。

透過 SSH 遠端存取 Cloud Shell

您也可以透過本機電腦遠端使用 Cloud Shell 功能。這通常涉及兩種不同的用途：

- 從本機終端機以 SSH 連線至 Cloud Shell 電腦
- 在本地掛接 Cloud Shell 遠端 `$HOME` 目錄。

為此，您需要在本地安裝 Google Cloud SDK。您也需要使用專案 ID 和所用的特定憑證進行設定。

在本機電腦上執行的終端機模擬器中，完成下列步驟：

1. 設定 GCP 專案 ID，並使用 Cloud Shell 所在 Cloud 機構的對應憑證登入。

```
gcloud config set project <your project id>
gcloud auth login
```

2. 透過 SSH 連線至遠端 Cloud Shell 機器：

```
gcloud cloud-shell ssh --authorize-session
```

您現在位於 Cloud Shell，但使用的是本機終端機模擬器的功能和設定。如果您使用的終端機支援 tmux，可以進一步整合，獲得更優質的遠端體驗。

從本機電腦掛接 Cloud Shell 主目錄

雖然能透過 SSH 連線至遠端 Cloud Shell 執行個體很不錯，但如果本機 IDE 可以存取遠端 Cloud Shell 的 `$HOME` 目錄，那就更棒了。這樣一來，您就能結合先前示範的 SSH 存取權，以及在本機編輯遠端程式碼的功能。

如要執行這項操作，請先在本機電腦上建立掛接點：

```
mkdir $HOME/cloudshell
```

這是 Cloud Shell 的掛接目錄。現在要掛接磁碟，請確認 Cloud Shell 執行個體已啟動，然後在本地終端機中輸入：

```
$(gcloud cloud-shell get-mount-command $HOME/cloudshell)
cd $HOME/cloudshell
```

這會取得您需要發布的掛接指令，在本機掛接 Cloud Shell。您會看到 Cloud Shell 主目錄的內容已掛接至本機。

現在，您可以在本機開啟 VSCode 等 IDE，並開啟雲端中的遠端工作區。同時，您也可以透過 SSH 存取 Cloud Shell，在 IDE 中開啟「終端機」窗格，將遠端終端機整合到本機 IDE 中。

步驟 5 · 約 0 分鐘

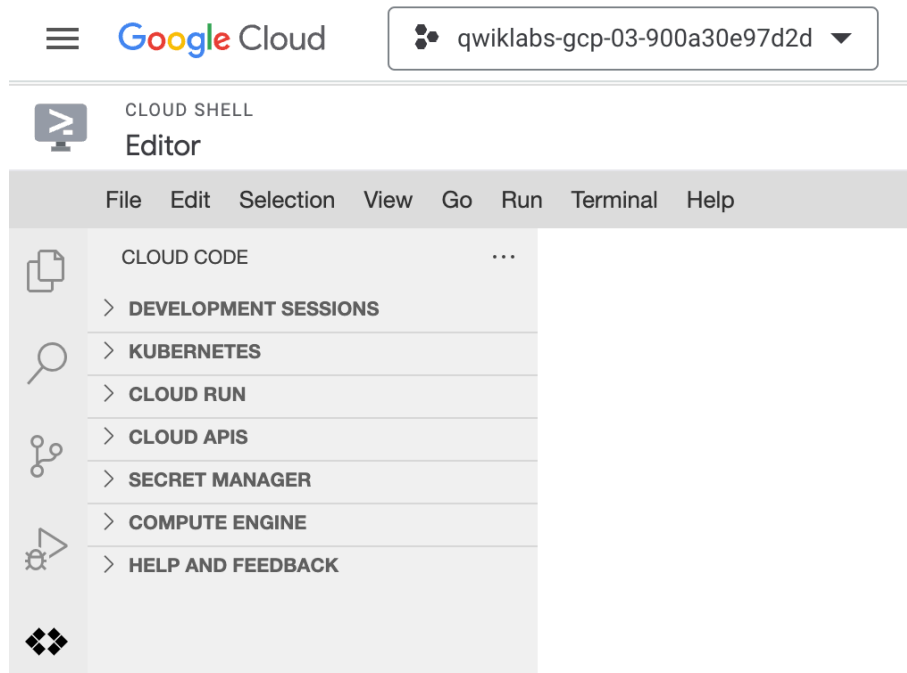
5. 使用 Cloud Code

Cloud Code 是 Google 開發的外掛程式，可協助開發人員更有效率地使用雲端工具。這項功能支援多種 IDE 和程式碼編輯器，例如 VSCode 和 JetBrains 產品，且預設會整合至 Cloud Shell 編輯器，方便您使用。Cloud Code 包含許多開發人員友善功能，您將在後續步驟中練習使用。

在 Cloud Shell 編輯器中尋找 Cloud Code

使用快速存取按鈕

在編輯器的左側窗格中，找出下列四個按鈕：



透過這些選單項目，您可以直接從 Cloud Shell 編輯器輕鬆存取及設定 GCP 服務。

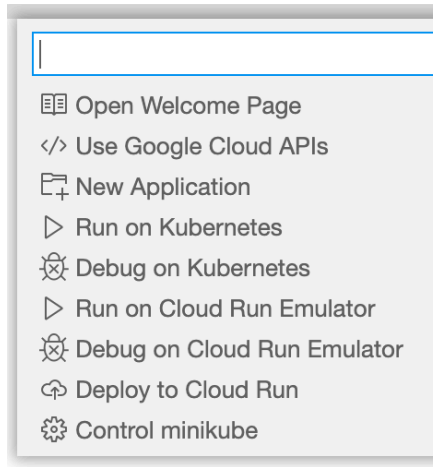
本實驗室的重點是 **Kubernetes 叢集**。

使用狀態列

透過 Cloud Shell 編輯器 UI 公開的 Cloud Code 還有兩個相關部分。您可以在編輯器的底部工具列中看到這兩項設定：



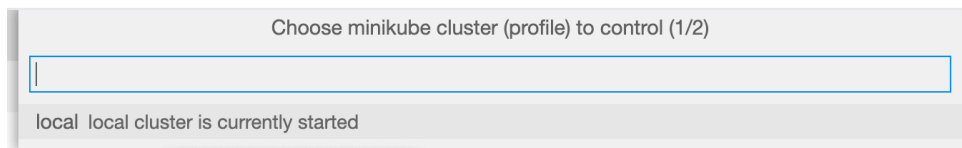
- **<> Cloud Code**：按一下這裡會顯示快速動作選單，您將在本教學課程中使用這個選單：



- 控制 minikube：您可以使用基本動作 (例如啟動或停止叢集)，控制本機 Kubernetes 模擬器 minikube。

建立 Minikube 執行個體

按一下按鈕，立即建立 Minikube 執行個體。



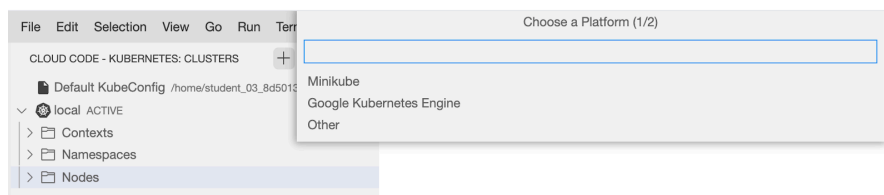
使用 Cloud Code 建立 GKE 叢集

1. 按一下左側的圖示 **Cloud Code - Kubernetes Clusters** (



)。左側會顯示名為「CLOUD CODE - KUBERNETES: CLUSTERS」的新窗格。

2. 按一下「**CLOUD CODE - KUBERNETES: CLUSTERS**」旁的 **+** 按鈕，然後選取「**Google Kubernetes Engine**」(請注意，您也可以選擇其他選項，例如先前啟動的 **Minikube**)：



3. 按一下「**建立新的 GKE 叢集**」。選取這個選項後，右側會載入新窗格，您可以在其中輸入額外資訊，快速建立開發叢集。按照面板中的指示操作，並輸入下列資訊：

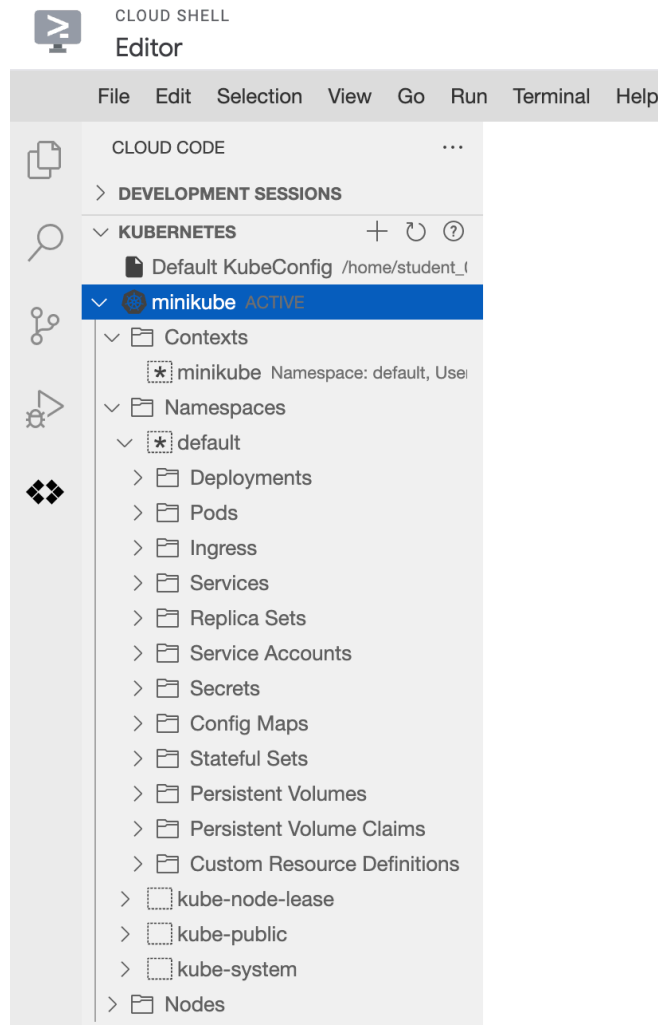
- 選擇 **Autopilot**
- 選取區域 (**europe-west-1**)
- 將叢集命名為「**dev**」

4. 按一下「**建立叢集**」按鈕。系統會建立新的 Autopilot 叢集。

叢集建立程序最多可能需要 5 分鐘。因此，在叢集建立期間，請繼續探索 Cloud Code Kubernetes 窗格。

探索 Cloud Code Kubernetes 窗格

您先前已建立 Minikube 叢集。準備就緒後，這個叢集會顯示在 Cloud Code Kubernetes 窗格中，名稱為您指定的名稱，`minikube`：



叢集會在 UI 中顯示為「ACTIVE」。這項設定會與指令列中的目前 Kubernetes 環境保持同步。。

您可以在 Cloud Code Kubernetes 窗格中看到下列內容：

- **預設 KubeConfig 檔案：**Cloud Code 會讀取 `~/ .kube/config` 檔案，並用來連線至設定的 Kubernetes 叢集，方便您輕鬆瀏覽。如要編輯 Kubeconfig 檔案，請將游標懸停在 `Default Kubeconfig` 行上，然後點選旁邊帶有向外箭頭的方塊。
- **可供瀏覽的已註冊叢集物件：**在本例中，您可以看到先前建立的 `minikube` Minikube 叢集，以及該叢集的內容、命名空間和節點。請展開樹狀結構中的部分節點。由於您尚未部署任何應用程式，因此目前不會有任何 Pod。

新的 GKE 叢集啟動並執行後，會與 `minikube` Minikube 叢集一起顯示在左側窗格中。請注意，您可以在任何叢集上按一下滑鼠右鍵，然後選取叢集來執行任何 Cloud Code 動作，讓叢集「處於啟用狀態」：



CLOUD SHELL

Editor

File Edit Selection View Go Run Terminal Help



CLOUD CODE

...

> DEVELOPMENT SESSIONS



✓ KUBERNETES

Default KubeConfig /home/student_1



> minikube ACTIVE



gke_qwiklabs-gcp-03-900a30e

Set as Active Cluster

Copy Value

Grant private GKE nodes outbound internet access

Set up Kubectl Proxying for cluster

Remove Context Under Cluster

步驟 6 · 約 0 分鐘

6. 恭喜！

恭喜，您已完成本程式碼研究室！

涵蓋內容

- 瀏覽並熟悉 Cloud Shell 的主要功能
 - 練習使用不同的 Cloud Shell 模式
 - 自訂 Cloud Shell 環境以供進階使用
 - 熟悉 Cloud Code 選項和功能
 - 查看 Kubernetes 應用程式適用的 Cloud Code 詳細資料
 - 使用 Minikube 等本機模擬器
-